
Autonomous Systems and Mobile Robots

Exercise Sheet 2 – Summer Term 2026

Issued: 06.05.2026

Professor

Prof. Dr. Malte Schilling
mschilling@uni-muenster.de

Tutor

Janosch Bajorath
j.bajorath@uni-muenster.de

Building a Robot Body — URDF, TF, and Gazebo Simulation

In this exercise you build a differential-drive robot from scratch using ROS 2 and the Unified Robot Description Format (URDF). You will organise your code in three packages, describe the robot geometry and mass properties, visualise it in RViz, simulate it in Gazebo, and finally drive it along a programmed trajectory — observing where open-loop control breaks down.

What you will build: a box-shaped chassis on two motorised wheels and a passive caster, described in XML, displayed in RViz with a hand-crafted configuration file, and running in Gazebo under a `/cmd_vel` velocity command interface.

What carries over: the three packages you create today will be extended in every subsequent session — sensors in Week 3, and the full robot platform for MiniLab 1 starting in Week 4.

Task 1 – Create the Workspace

You will build three packages that each have a clearly bounded responsibility. Understanding *why* code is split this way is as important as the split itself.

`my_robot_description`

Single source of truth for the robot model — URDF geometry and mass properties. Every other package reads from this one; nothing writes back to it.

`my_robot_bringup`

Integration layer — launch files, Gazebo worlds, bridge configuration, and RViz config. Knows how to assemble all pieces but contains no robot logic.

`my_robot_control`

Behaviour layer — the only package that publishes `/cmd_vel`. Depends on the running system but does not care how it was launched.

(a) **Workspace and package structure.**

Create a ROS 2 workspace with a `src/` directory. Inside `src/`, create three packages:

- `my_robot_description` — build type `ament_cmake`
- `my_robot_bringup` — build type `ament_cmake`; declares `my_robot_description` as a dependency
- `my_robot_control` — build type `ament_python`; depends on `rclpy` and `geometry_msgs`

Convention. `ros2 pkg create` accepts `-build-type` and `-dependencies`.

(b) **Directory structure.**

Inside each package, create the subdirectories it will need. The resulting layout should match the following tree:

```
ros2_ws/
|-- src/
|   |-- my_robot_description/
|   |   |-- urdf/
|   |   |-- meshes/
|   |-- my_robot_bringup/
|   |   |-- launch/
|   |   |-- worlds/
|   |   |-- config/
|   |-- my_robot_control/
|       |-- my_robot_control/
|       |-- scripts/
```

Convention. For `ament_cmake` packages, install directories in `CMakeLists.txt` with: `install(DIRECTORY ... DESTINATION share/${PROJECT_NAME})`.

(c) **Build and verify.**

Build all three packages with `colcon build -symlink-install` and source the install overlay. Confirm that `ros2 pkg list` shows all three.

Convention. Source `install/setup.bash` so that `ros2` commands can find the newly built packages.

Task 2 – Describe the Chassis

A URDF (Unified Robot Description Format) file is an XML document that describes a robot as a tree of *links* connected by *joints*. Every physical body is a link; every connection between two bodies is a joint. In this task you write the root link of your robot.

Formula. Inertia of a uniform box (mass m , side lengths $l \times w \times h$ along x, y, z):

$$I_{xx} = \frac{m}{12}(w^2 + h^2), \quad I_{yy} = \frac{m}{12}(l^2 + h^2), \quad I_{zz} = \frac{m}{12}(l^2 + w^2).$$

Use a realistic chassis mass (e.g. 5 kg). Off-diagonal terms are zero for a symmetric, uniform body.

(a) **Create the robot base.**

In `my_robot_description/urdf/`, create `my_robot.urdf.xacro`. The root XML element must be `<robot name="my_robot">` with the xacro namespace declared. Define a single link called `base_link` representing a box-shaped chassis of your choice (suggested: $0.30 \times 0.20 \times 0.10$ m).

File skeleton. Every Xacro file starts with an XML declaration and a `<robot>` root element that declares the Xacro namespace:

```
<?xml version="1.0"?>
<robot name="my_robot"
  xmlns:xacro="http://ros.org/wiki/xacro">

  <!-- links and joints go here -->

</robot>
```

Links and joints are nested inside `<robot>`.

Each link can carry three sub-elements:

- `<visual>` — geometry and colour for rendering
- `<collision>` — geometry used by the physics engine for contact detection (often simplified)
- `<inertial>` — mass and inertia tensor, required by Gazebo for dynamics

Syntax. `snippets/urdf_reference.urdf.xacro` shows the full link structure for all three geometry types — open it alongside this exercise and refer to the **BOX LINK** section.

(b) **Verify the URDF parses.**

Process the file with the `xacro` command-line tool and inspect the output XML. Confirm the link and all three sub-elements appear.

Command. `xacro my_robot.urdf.xacro` prints the resolved XML to stdout. Pipe to `less` or redirect to a `.urdf` file for easier reading.

Task 3 – Add the Drive Wheels

Joints connect links. A **continuous** joint allows unlimited rotation around a fixed axis — the right type for a motorised wheel. A joint's `<origin>` defines where the child frame sits relative to the parent frame; the `<axis>` defines which direction the joint rotates around.

Formula. Inertia of a solid cylinder (mass m , radius r , length l):

$$I_{xx} = I_{yy} = \frac{m}{12}(3r^2 + l^2), \quad I_{zz} = \frac{mr^2}{2}.$$

(a) **Add wheel links.**

Add two cylinder links: `left_wheel_link` and `right_wheel_link`. Choose a wheel radius of 0.05 m and width of 0.03 m. Include `<visual>`, `<collision>`, and `<inertial>` elements.

Syntax. `snippets/urdf_reference.urdf.xacro` shows the full link structure for all three geometry types — refer to the **CYLINDER LINK** section.

Axis orientation. In URDF the cylinder axis is along the local z direction. To make the wheel spin around the world y axis, rotate the link frame by $-\pi/2$ around x : `<origin rpy="-1.5708 0 0"/>`. After this rotation the local z axis aligns with y , so the joint axis is `<axis xyz="0 0 1"/>`.

(b) **Add wheel joints.**

Connect each wheel to `base_link` with a **continuous** joint. Place the left wheel at $y = +0.13$ m and the right wheel at $y = -0.13$ m, both at $x = +0.05$ m, $z = -0.05$ m. The x offset puts the axle slightly in front of the chassis centre so the robot's centre of mass falls inside the support triangle formed by the two drive wheels and the rear caster, keeping the robot stable. The $z = -0.05$ m places the joint at the bottom face of the body, because `base_link` sits at the body's geometric centre.

Derivation: Chassis half-height = 0.05 m, so `base_link` sits $0.05 + 0.05 = 0.10$ m above the ground (half-height plus wheel radius). The wheel centre must be at wheel radius = 0.05 m above the ground. Working down from `base_link`: $0.10 - 0.05 = 0.05$ m above ground, so the joint offset is $z = -0.05$ m. If you change the chassis height or wheel radius, recompute this value.

(c) **Add joint dynamics.**

Inside each wheel joint, add a `<dynamics>` element with `damping="0.1"` and `friction="0.1"`. This provides resistance that would otherwise be absent in the frictionless ideal joint model, narrowing the sim-to-real gap.

(d) **Verify.**

Process the URDF with `xacro` and count the links and joints. You should have three links and two joints at this point.

Task 4 – Add the Caster Wheel

A differential-drive robot with only two wheels is statically unstable — it needs a third contact point. A passive caster provides this: it does not drive or steer, but freely swivels. In our simplified model a fixed sphere is sufficient.

Formula. Inertia of a solid sphere (mass m , radius r):

$$I_{xx} = I_{yy} = I_{zz} = \frac{2}{5}mr^2.$$

(a) **Add the caster link.**

Add a link called `caster_link` with sphere geometry, radius 0.025 m. Include visual, collision, and inertial elements.

Syntax. `snippets/urdf_reference.urdf.xacro` shows the full link structure for all three geometry types — refer to the **SPHERE LINK** section.

(b) **Add the caster joint.**

Connect `caster_link` to `base_link` with a **fixed** joint. Place the sphere so that its bottom just touches the ground when the chassis sits at wheel-axle height.

Hint. `base_link` is at the geometric centre of the chassis body, 0.10 m above the ground (wheel-axle height + body half-height). The caster sphere (radius 0.025 m) must touch the ground, so its centre sits 0.025 m above the ground. Working upward from the ground: $z = 0.025 - 0.10 = -0.075$ m from `base_link`. Place it at the rear of the chassis ($x = -0.13$ m).

(c) **Verify.**

Process the URDF with `xacro` and confirm you now have four links and three joints.

Task 5 – Visualise in RViz

`robot_state_publisher` reads the URDF from the `/robot_description` parameter and publishes the coordinate frames of every link as TF transforms. RViz subscribes to these transforms to render the robot in 3D.

(a) **Write the visualisation launch file.**

In `my_robot_bringup/launch/`, write `robot_vis.launch.py`. It should start three nodes: `robot_state_publisher` (with the URDF loaded from the description package via `xacro`), `joint_state_publisher_gui` (so you can move wheel joints interactively), and `rviz2` (pointed at your config file).

Convention. Import the substitution utilities at the top of the launch file:

```
from launch.substitutions import FindPackageShare, PathJoinSubstitution
```

Use `FindPackageShare` and `PathJoinSubstitution` to build paths that resolve at launch time, not at write time.

Hint. Pass the URDF to `robot_state_publisher` as: `parameters=[{'robot_description': Command(['xacro ', urdf_path])}]`. Pass the RViz config with `arguments=['-d', rviz_config]`.

(b) **Configure RViz using the GUI.**

With `robot_vis.launch.py` running and RViz open, build your visualisation step by step.

1. **Set the Fixed Frame.** In the *Global Options* panel on the left, change **Fixed Frame** from `map` to `base_link`. This tells RViz which coordinate frame to treat as the origin of the scene.
2. **Add a RobotModel display.** Click **Add** (bottom-left) → *By display type* → `rviz_default_plugins/RobotModel`. In the display's properties, set **Description Topic** to `/robot_description`. The robot geometry should appear in the 3D view.
3. **Add a TF display.** Click **Add** → *By display type* → `rviz_default_plugins/TF`. Enable **Show Names** in the display properties to label each frame.
4. **Save the configuration.** **File** → **Save Config As**, navigate to `my_robot_bringup/config/`, and save as `my_robot.rviz`. This file is what the launch file references via the `-d` argument.

Concept. RViz displays can be added *by display type* (you choose the plugin, then set the topic) or *by topic* (RViz infers the display type from the message type). Both paths lead to the same result — use whichever is easier to navigate.

(c) **Launch and verify.**

Build both packages, source the overlay, and launch `robot_vis.launch.py`. Confirm:

- The robot appears in RViz with correct geometry
- Moving the `joint_state_publisher_gui` sliders rotates the wheels
- No red *No transform from ...* errors appear in RViz

Task 6 – Inspect the TF Tree

The TF tree is the runtime representation of the robot's kinematic chain. Every joint in the URDF becomes a transform in the TF tree, published continuously by `robot_state_publisher`.

(a) **Generate the TF tree diagram.**

While `robot_vis.launch.py` is running, generate a PDF of the TF tree:

```
ros2 run tf2_tools view_frames
```

Open the resulting `frames.pdf`. How many frames does your robot have? Identify the root frame and all leaf frames.

(b) **Trace the kinematic chain.**

For each of the following frames, state its parent frame and the joint type that connects them: `left_wheel_link`, `right_wheel_link`, `caster_link`.

(c) **Observe joint motion in TF.**

With RViz's TF display enabled, move the wheel sliders in `joint_state_publisher_gui`. What changes in the 3D view? What stays fixed?

(d) **Compare visualisation mode with simulation mode.**

You will add Gazebo simulation in Task 7. Before doing so, answer: which frame is present in simulation mode but absent in your current visualisation-only TF tree? What publishes that transform in simulation? What publishes it in a real robot?

Convention. Think about the transform that describes the robot's position in the world. In simulation this comes from the diff-drive controller plugin. On a real robot it typically comes from wheel encoder odometry.

Task 7 – Prepare the Gazebo Simulation Files

Gazebo needs four files before the simulation can run: a plugin description file (`robot.gazebo`), an include directive in the main URDF, a world description (`my_world.sdf`), and a bridge configuration (`bridge.yaml`). You write all four here; the launch file that assembles them comes in Task 8.

Starter files. The `snippets/` folder on the course repository contains starter files for this task: `robot.gazebo`, `bridge.yaml`, and `my_world.sdf`. Copy them into your workspace and complete the TODO items as you work through the subtasks below.

(a) **Complete `urdf/robot.gazebo`.**

Copy `snippets/robot.gazebo` to `my_robot_description/urdf/robot.gazebo`. The diff-drive plugin block is already provided — update `wheel_separation` to match your URDF (centre-to-centre distance of the two wheel joints along y), then complete the three remaining TODO blocks:

- **Joint state publisher plugin** (`gz-sim-joint-state-publisher-system`): replaces `joint_state_publisher_gui` in simulation — Gazebo publishes the actual joint angles.
- **Drive wheel surface properties:** one `<gazebo reference="...">` block per wheel link, `mu1` and `mu2` set to 1.0.
- **Caster surface:** same structure, `mu1` and `mu2` set to 0.0 so the caster does not resist lateral motion.

Convention. The `<tf_topic>/tf</tf_topic>` line in the diff-drive plugin is important: it ensures the plugin publishes the `odom`→`base_link` transform to the `/tf` topic that the bridge forwards to ROS2.

(b) **Include `robot.gazebo` in the main URDF.**

At the top of `my_robot.urdf.xacro` (before the first link), add a `<xacro:include>` directive:

Syntax.

```
<xacro:include
  filename="$(find my_robot_description)/urdf/robot.gazebo"/>
```

(c) **Copy `worlds/my_world.sdf`.**

Copy `snippets/my_world.sdf` to `my_robot_bringup/worlds/my_world.sdf`. No edits needed — the SDF world format is not a learning objective this week.

(d) **Complete `config/bridge.yaml`.**

Copy `snippets/bridge.yaml` to `my_robot_bringup/config/bridge.yaml`. The `/cmd_vel` entry is already provided; complete the four remaining TODO entries. Each entry bridges one topic between Gazebo and ROS2 and requires five fields: `ros_topic_name`, `gz_topic_name`, `ros_type_name`, `gz_type_name`, and `direction`. The snippet file contains the type names and directions as inline comments.

Task 8 – Launch the Gazebo Simulation

With the simulation files in place, you now write the launch file that assembles them, verify the system runs end-to-end, and extend your RViz configuration to visualise odometry.

(a) **Write `robot_sim.launch.py`.**

In `my_robot_bringup/launch/`, write `robot_sim.launch.py`. It should:

- Start `robot_state_publisher` (same as in `robot_vis.launch.py`)
- Launch Gazebo with your world file
- Spawn the robot from the `/robot_description` topic
- Start the bridge with your YAML config
- Open RViz with your saved config

Code. Launch Gazebo with `ExecuteProcess` (import from `launch.actions`):

```
ExecuteProcess(cmd=['gz', 'sim', '-r', world_path], output='screen')
```

Spawn the robot: `package ros_gz_sim, executable create, arguments ['-topic', 'robot_description', '-name', 'my_robot', '-z', '0.1']`.
Start the bridge: `package ros_gz_bridge, executable parameter_bridge, with parameter config_file pointing to bridge.yaml.`

(b) **Verify and extend RViz.**

Build, source, and launch `robot_sim.launch.py`. Confirm:

- Gazebo opens with the flat world and the robot spawns
- `ros2 topic list` shows `/cmd_vel`, `/odom`, `/joint_states`, `/tf`
- `ros2 run tf2_tools view_frames` now shows `odom` as the root frame

Then extend the RViz configuration for odometry:

1. **Switch the Fixed Frame** from `base_link` to `odom`. The robot model stays visible but the scene is now anchored to the world origin.
2. **Add an Odometry display.** Click **Add** → *By topic* → `/odom` → `Odometry`. This shows the robot's estimated pose as an arrow in the `odom` frame.
3. **Drive the robot.** In a second terminal:

```
ros2 run teleop_twist_keyboard teleop_twist_keyboard
```

Drive forward, turn, and return roughly to the start position. Watch the odometry arrow in RViz.

4. **Discussion.** After driving a full circle, does the robot's reported position in RViz match its actual position in Gazebo? What would cause a discrepancy on a real robot?

5. **Save the updated config.** **File** → **Save Config** to overwrite `my_robot.rviz` with the new Odometry display.

Task 9 – Drive and Observe

An open-loop controller sends commands to the robot but reads no feedback. It assumes the robot executes every command perfectly — an assumption that is never fully true in the real world. In this task you write an open-loop square trajectory, observe when it works, and then deliberately break the robot’s model to see why it fails.

(a) **Create `my_robot_control`.**

In `my_robot_control/my_robot_control/`, create `movement.py`. Define two functions:

- `drive(publisher, linear_x, angular_z, duration_s)` — publishes a `geometry_msgs/Twist` with the given velocities, then blocks for `duration_s` seconds using `time.sleep()`.
- `stop(publisher)` — publishes a zero `Twist`.

Neither function creates a ROS node; they receive a publisher as an argument. This keeps the module testable and reusable across scripts.

Code. Import from `geometry_msgs.msg` import `Twist`. A zero `Twist` is simply `Twist()` with no field assignments — all fields default to 0.0.

(b) **Write `move_square.py`.**

In the same module directory, create `move_square.py` with a `main()` function. It should: create a `rclpy` node and a publisher on `/cmd_vel`, wait briefly for DDS discovery (`time.sleep(2.0)`), then call `drive` and `stop` from `movement` to drive four sides of a square, and finally shut down cleanly.

Choose forward speed and duration to drive roughly 0.6 m per side. Choose angular speed and duration to turn roughly 90 per corner.

Convention. Register the entry point in `setup.py` under `console_scripts`: `'move_square = my_robot_control.move_square:main'`. After `colcon build`, `ros2 run my_robot_control move_square` will find it.

Code. There is no need to subclass `Node` — `rclpy.create_node()` is sufficient for a publisher-only script.

(c) **Run the baseline.**

With `robot_sim.launch.py` running in one terminal, build and run `move_square` in a second terminal. Set the RViz fixed frame to `odom` and watch the `odom`→`base_link` transform. Does the robot return to its start position?

In simulation with ideal physics and correct model parameters the answer should be yes — this is the *best case*.

(d) **Introduce a model parameter error.**

Open `robot.gazebo`. The differential-drive plugin uses `wheel_separation` to convert a commanded angular velocity into individual wheel speeds. This value was measured from your URDF — but in practice you measure it with a ruler and get it slightly wrong.

Change `wheel_separation` from its correct value to 0.28 (about 8% too large), rebuild, and run `move_square` again.

- How does the trajectory change?
- Does the error grow with each corner, or stay constant?
- Run `ros2 topic echo /odom` after the trajectory completes. What is the reported heading error?

Restore the correct value when done.

(e) **Introduce asymmetric friction.**

In `robot.gazebo`, set `mu1` and `mu2` to 0.25 for one wheel link only (leave the other at 1.0), rebuild, and run the square again.

- In which direction does the robot veer?
- Is the drift pattern the same as with the wheel-separation error, or different?

Restore symmetric friction when done.